

RESULTADO DE LA CORRECCIÓN: **DESAPROBADO**

OBSERVACIONES

Ejercicio 4: acepta listas de diferentes longitudes. No siempre funciona correctamente.

```

02-trabajoPractico-Listas > C main.c > ejercicio6()
381 void ejercicio6(){
390     while(seguirAgregando){
396     }else{
400     }else{
403
404         fflush(stdin);
405     }
406 }
407
408 seguirAgregando = true;
409 index = 1;
410
411 while(seguirAgregando){
412     printf("Ingrese el #%i elemento de la segunda lista o 'n' para terminar: ", index);
413
414     if(scanf("%d", &inputNumber) > 0){
415         l_agregar(L2, te_crear(inputNumber));

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

-- 4. COMPARAR LISTAS (MAYOR, IGUAL O MENOR) --
Si desea llenar las listas manualmente presione '1', de lo contrario presione '0' (Auto):
Ingrese un numero entre 0 y 1: 1

Ingrese cantidad de elementos para la Lista 1:
Ingrese un numero entre 0 y 100: 0

Ingrese cantidad de elementos para la Lista 2:
Ingrese un numero entre 0 y 100: 2
Ingrese las claves que desea para sus TipoElementos:
Clave del elemento 1: 0
Clave del elemento 2: 1

Lista 1: Contenido de la lista:
Lista 2: Contenido de la lista: 0 1

>>> RESULTADO DE LA COMPARACION: 0
Interpretacion: Ambas listas son IGUALES.

Presione ENTER para volver al menu principal

```

```

02-trabajoPractico-Listas > C main.c > ejercicio6()
381 void ejercicio6(){
390     while(seguirAgregando){
396     }else{
400     }else{
403
404         fflush(stdin);
405     }
406 }
407
408 seguirAgregando = true;
409 index = 1;
410
411 while(seguirAgregando){
412     printf("Ingrese el #%i elemento de la segunda lista o 'n' para terminar: ", index);
413
414     if(scanf("%d", &inputNumber) > 0){
415         l_agregar(L2, te_crear(inputNumber));

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

-- 4. COMPARAR LISTAS (MAYOR, IGUAL O MENOR) --
Si desea llenar las listas manualmente presione '1', de lo contrario presione '0' (Auto):
Ingrese un numero entre 0 y 1: 1

Ingrese cantidad de elementos para la Lista 1:
Ingrese un numero entre 0 y 100: 0

Ingrese cantidad de elementos para la Lista 2:
Ingrese un numero entre 0 y 100: 2
Ingrese las claves que desea para sus TipoElementos:
Clave del elemento 1: 1
Clave del elemento 2: 2

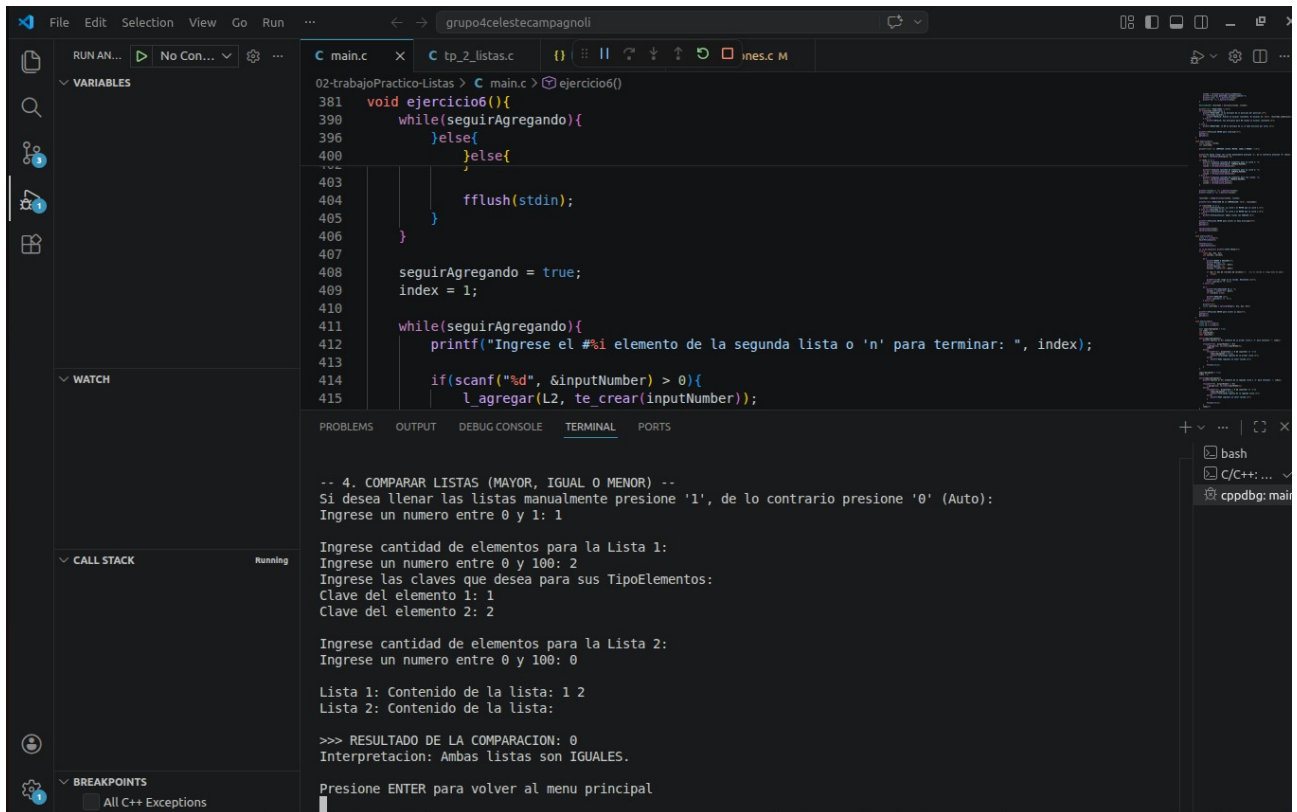
Lista 1: Contenido de la lista:
Lista 2: Contenido de la lista: 1 2

>>> RESULTADO DE LA COMPARACION: 0
Interpretacion: Ambas listas son IGUALES.

Presione ENTER para volver al menu principal

```

GRUPO 4 – Correcciones Trabajo Práctico: LISTAS



```
02-trabajoPractico-Listas > C main.c > ejercicio6()
381 void ejercicio6(){
390     while(seguirAgregando){
396     }else{
400     }
403
404         fflush(stdin);
405     }
406 }
407
408     seguirAgregando = true;
409     index = 1;
410
411     while(seguirAgregando){
412         printf("Ingrese el #%i elemento de la segunda lista o 'n' para terminar: ", index);
413
414         if(scanf("%d", &inputNumber) > 0){
415             l_agregar(L2, te_crear(inputNumber));

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

-- 4. COMPARAR LISTAS (MAYOR, IGUAL O MENOR) --
Si desea llenar las listas manualmente presione '1', de lo contrario presione '0' (Auto):
Ingrese un numero entre 0 y 1: 1

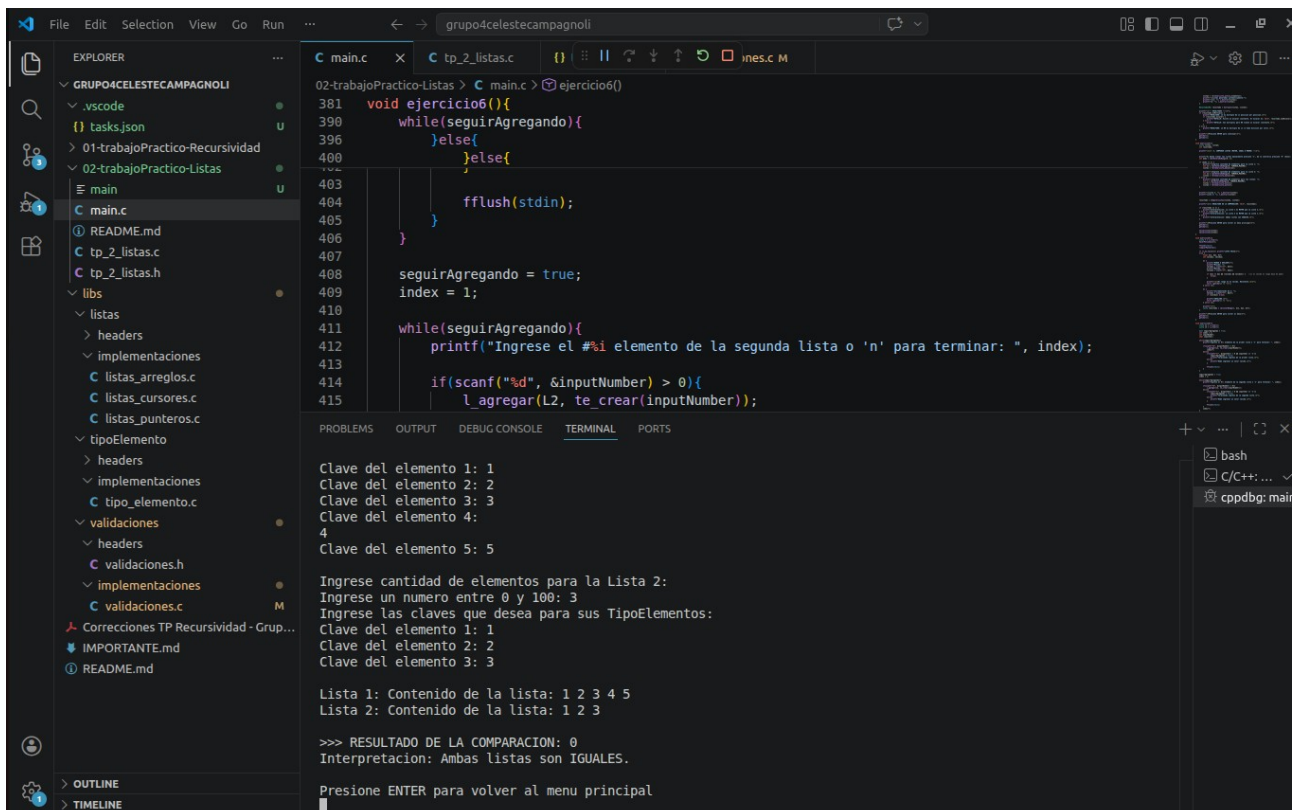
Ingrese cantidad de elementos para la Lista 1:
Ingrese un numero entre 0 y 100: 2
Ingrese las claves que desea para sus TipoElementos:
Clave del elemento 1: 1
Clave del elemento 2: 2

Ingrese cantidad de elementos para la Lista 2:
Ingrese un numero entre 0 y 100: 0

Lista 1: Contenido de la lista: 1 2
Lista 2: Contenido de la lista:

>>> RESULTADO DE LA COMPARACION: 0
Interpretacion: Ambas listas son IGUALES.

Presione ENTER para volver al menu principal



```
02-trabajoPractico-Listas > C main.c > ejercicio6()
381 void ejercicio6(){
390     while(seguirAgregando){
396     }else{
400     }
403
404         fflush(stdin);
405     }
406 }
407
408     seguirAgregando = true;
409     index = 1;
410
411     while(seguirAgregando){
412         printf("Ingrese el #%i elemento de la segunda lista o 'n' para terminar: ", index);
413
414         if(scanf("%d", &inputNumber) > 0){
415             l_agregar(L2, te_crear(inputNumber));

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Clave del elemento 1: 1
Clave del elemento 2: 2
Clave del elemento 3: 3
Clave del elemento 4: 4
Clave del elemento 5: 5

Ingrese cantidad de elementos para la Lista 2:
Ingrese un numero entre 0 y 100: 3
Ingrese las claves que desea para sus TipoElementos:
Clave del elemento 1: 1
Clave del elemento 2: 2
Clave del elemento 3: 3

Lista 1: Contenido de la lista: 1 2 3 4 5
Lista 2: Contenido de la lista: 1 2 3

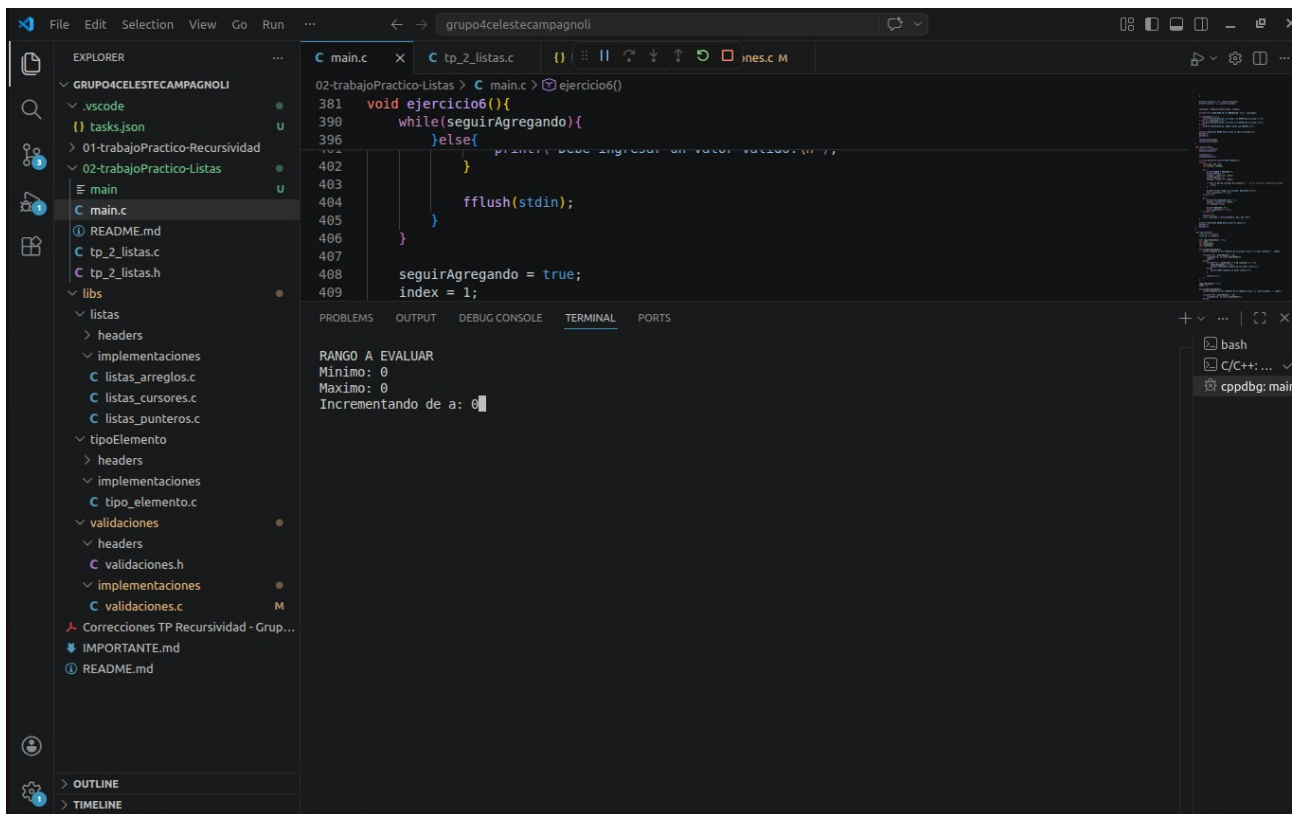
>>> RESULTADO DE LA COMPARACION: 0
Interpretacion: Ambas listas son IGUALES.

Presione ENTER para volver al menu principal

GRUPO 4 – Correcciones Trabajo Práctico: LISTAS

Ejercicio 5: si se ingresa 0 en todos los valores que pide, incluso el valor de salto, entra en loop. Pide los exponentes y no controla si se repiten.

GRUPO 4 – Correcciones Trabajo Práctico: LISTAS



```
File Edit Selection View Go Run ... grupo4celestecampagnoli
```

EXPLORER

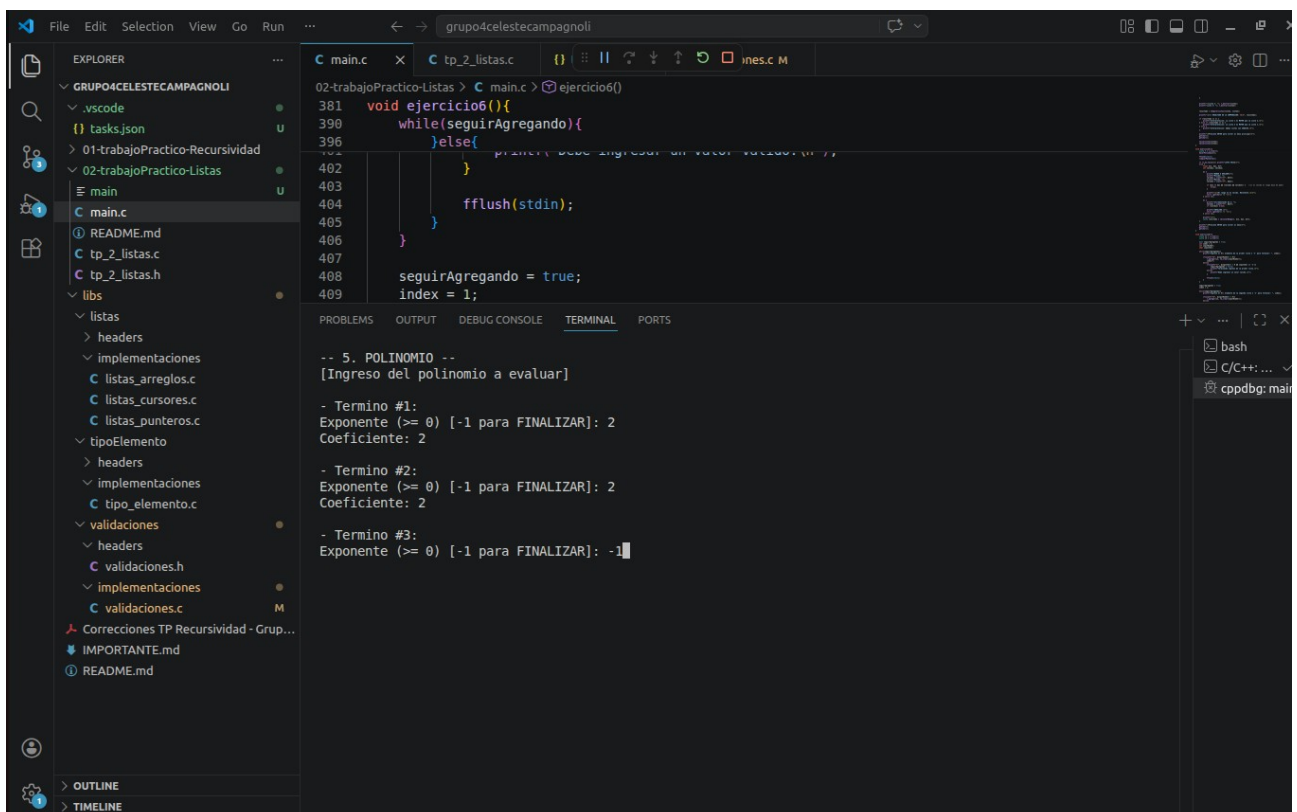
- GRUPO4CELESTECAMPAGNOLI
 - .vscode
 - tasks.json
 - 01-trabajoPractico-Recursividad
 - 02-trabajoPractico-Listas
 - main
 - main.c
 - README.md
 - tp_2_listas.c
 - tp_2_listas.h
 - libs
 - listas
 - headers
 - implementaciones
 - listas_arreglos.c
 - listas_cursoros.c
 - listas_punteros.c
 - tipoElemento
 - headers
 - implementaciones
 - tipo_elemento.c
 - validaciones
 - headers
 - implementaciones
 - validaciones.h
 - implementaciones
 - validaciones.c
 - Correcciones TP Recursividad - Grup...
 - IMPORTANTE.md
 - README.md

main.c

```
02-trabajoPractico-Listas > C main.c > ejercicio6()
381 void ejercicio6(){
390 while(seguirAgregando){
396 }else{
402 }
403 }
404 fflush(stdin);
405 }
406 }
407 }
408 seguirAgregando = true;
409 index = 1;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

RANGO A EVALUAR
Minimo: 0
Maximo: 0
Incrementando de a: 0



```
File Edit Selection View Go Run ... grupo4celestecampagnoli
```

EXPLORER

- GRUPO4CELESTECAMPAGNOLI
 - .vscode
 - tasks.json
 - 01-trabajoPractico-Recursividad
 - 02-trabajoPractico-Listas
 - main
 - main.c
 - README.md
 - tp_2_listas.c
 - tp_2_listas.h
 - libs
 - listas
 - headers
 - implementaciones
 - listas_arreglos.c
 - listas_cursoros.c
 - listas_punteros.c
 - tipoElemento
 - headers
 - implementaciones
 - tipo_elemento.c
 - validaciones
 - headers
 - implementaciones
 - validaciones.h
 - implementaciones
 - validaciones.c
 - Correcciones TP Recursividad - Grup...
 - IMPORTANTE.md
 - README.md

main.c

```
02-trabajoPractico-Listas > C main.c > ejercicio6()
381 void ejercicio6(){
390 while(seguirAgregando){
396 }else{
402 }
403 }
404 fflush(stdin);
405 }
406 }
407 }
408 seguirAgregando = true;
409 index = 1;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

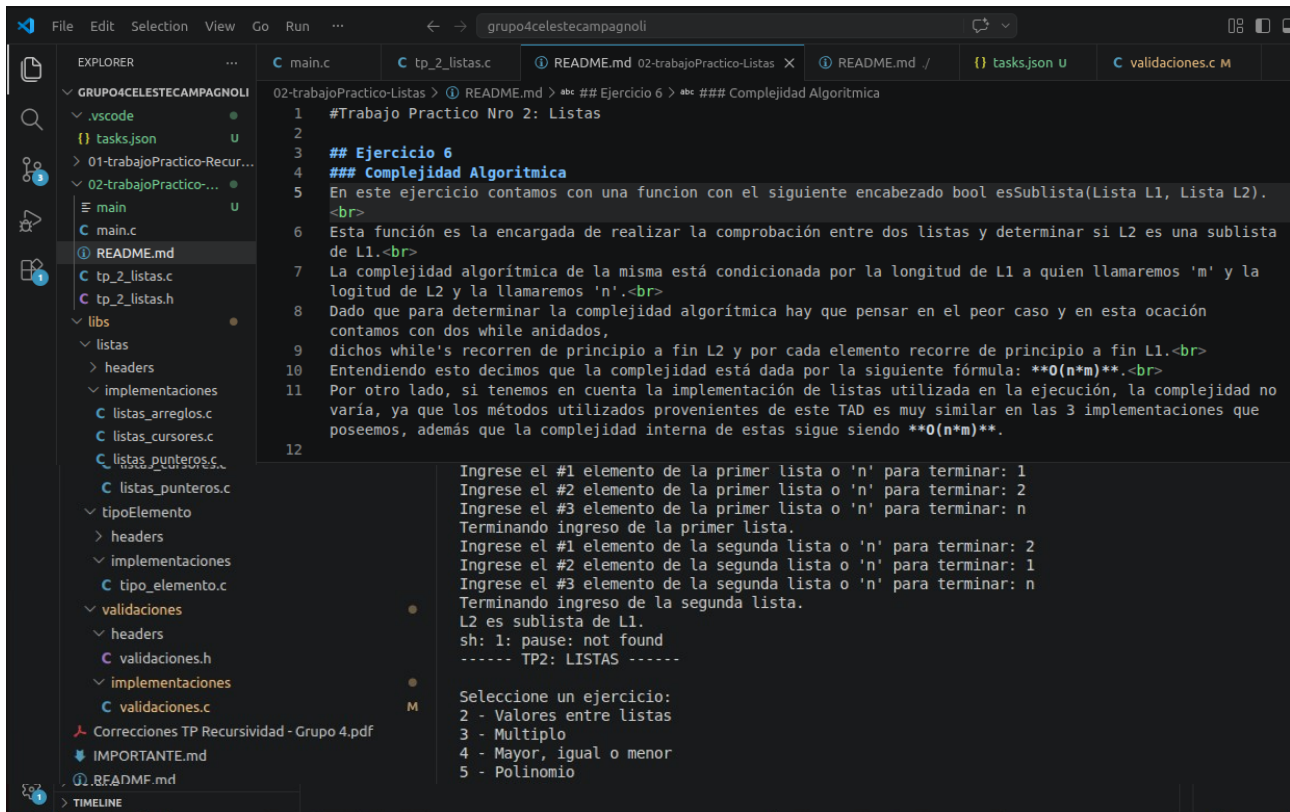
-- 5. POLINOMIO --
[Ingreso del polinomio a evaluar]

- Termino #1:
Exponente (>= 0) [-1 para FINALIZAR]: 2
Coeficiente: 2

- Termino #2:
Exponente (>= 0) [-1 para FINALIZAR]: 2
Coeficiente: 2

- Termino #3:
Exponente (>= 0) [-1 para FINALIZAR]: -1

GRUPO 4 – Correcciones Trabajo Práctico: LISTAS

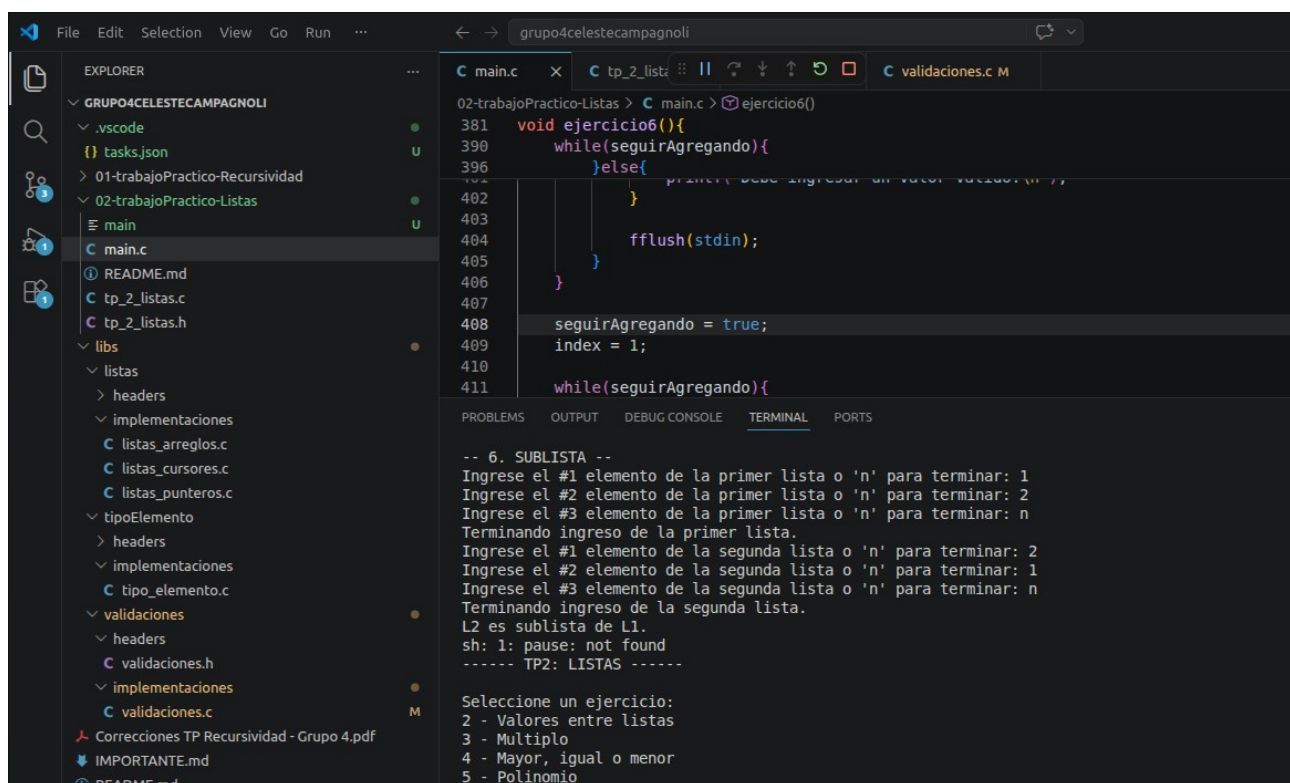


```
1 #Trabajo Practico Nro 2: Listas
2
3 ## Ejercicio 6
4 ## Complejidad Algorítmica
5 En este ejercicio contamos con una funcion con el siguiente encabezado bool esSublista(Lista L1, Lista L2).
6 Esta función es la encargada de realizar la comprobación entre dos listas y determinar si L2 es una sublista
  de L1.<br>
7 La complejidad algorítmica de la misma está condicionada por la longitud de L1 a quien llamaremos 'm' y la
  logitud de L2 y la llamaremos 'n'.<br>
8 Dado que para determinar la complejidad algorítmica hay que pensar en el peor caso y en esta ocasión
  contamos con dos while anidados,
9 dichos while's recorren de principio a fin L2 y por cada elemento recorre de principio a fin L1.<br>
10 Entendiendo esto decimos que la complejidad está dada por la siguiente fórmula: **O(n*m)**.<br>
11 Por otro lado, si tenemos en cuenta la implementación de listas utilizada en la ejecución, la complejidad no
  varía, ya que los métodos utilizados provenientes de este TAD es muy similar en las 3 implementaciones que
  poseemos, además que la complejidad interna de estas sigue siendo **O(n*m)**.
12
```

```
Ingrese el #1 elemento de la primer lista o 'n' para terminar: 1
Ingrese el #2 elemento de la primer lista o 'n' para terminar: 2
Ingrese el #3 elemento de la primer lista o 'n' para terminar: n
Terminando ingreso de la primer lista.
Ingrese el #1 elemento de la segunda lista o 'n' para terminar: 2
Ingrese el #2 elemento de la segunda lista o 'n' para terminar: 1
Ingrese el #3 elemento de la segunda lista o 'n' para terminar: n
Terminando ingreso de la segunda lista.
L2 es sublista de L1.
sh: 1: pause: not found
----- TP2: LISTAS -----

Seleccione un ejercicio:
2 - Valores entre listas
3 - Multiplo
4 - Mayor, igual o menor
5 - Polinomio
```

Ejercicio 6: compara en un solo sentido. La complejidad está incorrecta, respondo $O(m*n)$ y es $Q(n**2)$.

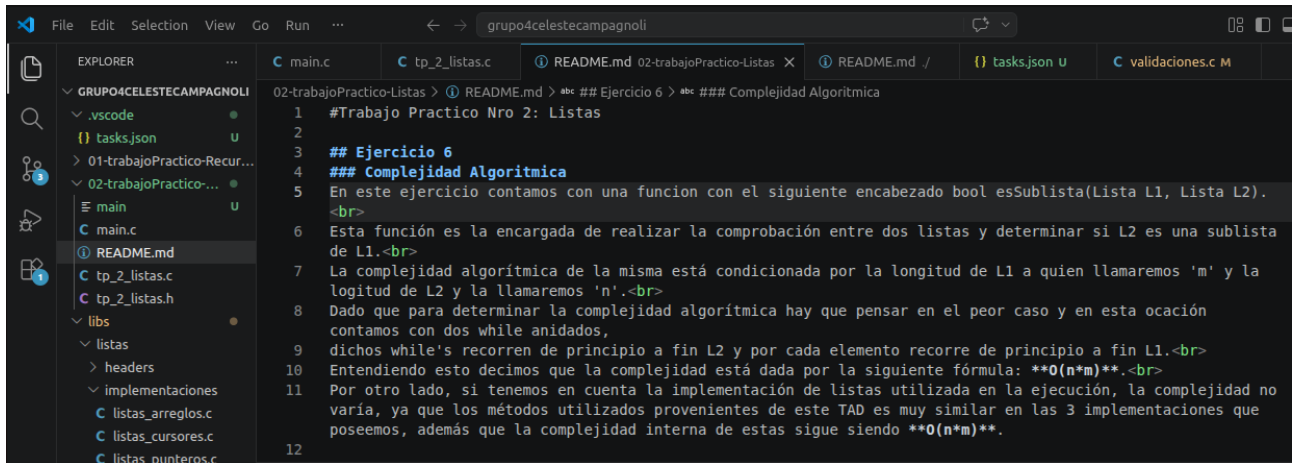


```
381 void ejercicio6(){
390     while(seguirAgregando){
396     }else{
402         printf("Debe ingresar un valor valido.\n");
403     }
404     fflush(stdin);
405 }
406
407 seguirAgregando = true;
408 index = 1;
409
410 while(seguirAgregando){
411
```

```
-- 6. SUBLISTA --
Ingrese el #1 elemento de la primer lista o 'n' para terminar: 1
Ingrese el #2 elemento de la primer lista o 'n' para terminar: 2
Ingrese el #3 elemento de la primer lista o 'n' para terminar: n
Terminando ingreso de la primer lista.
Ingrese el #1 elemento de la segunda lista o 'n' para terminar: 2
Ingrese el #2 elemento de la segunda lista o 'n' para terminar: 1
Ingrese el #3 elemento de la segunda lista o 'n' para terminar: n
Terminando ingreso de la segunda lista.
L2 es sublista de L1.
sh: 1: pause: not found
----- TP2: LISTAS -----

Seleccione un ejercicio:
2 - Valores entre listas
3 - Multiplo
4 - Mayor, igual o menor
5 - Polinomio
```

GRUPO 4 – Correcciones Trabajo Práctico: LISTAS



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left and the Editor view on the right. The Explorer sidebar shows the project structure for 'GRUPO4CELESTECAMPAGNOLI', including files like .vscode, tasks.json, 01-trabajoPractico-Recur..., 02-trabajoPractico-..., main.c, tp_2_listas.c, tp_2_listas.h, and a libs folder containing headers, implementaciones, and listas_arreglos.c, listas_cursor.c, and listas_punteros.c. The Editor view shows the README.md file for '02-trabajoPractico-Listas', which contains the following content:

```
1 #Trabajo Practico Nro 2: Listas
2
3 ## Ejercicio 6
4 ### Complejidad Algoritmica
5 En este ejercicio contamos con una funcion con el siguiente encabezado bool esSublista(Lista L1, Lista L2).
6 Esta función es la encargada de realizar la comprobación entre dos listas y determinar si L2 es una sublista
7 de L1.<br>
8 La complejidad algoritmica de la misma está condicionada por la longitud de L1 a quien llamaremos 'm' y la
9 longitud de L2 y la llamaremos 'n'.<br>
10 Dado que para determinar la complejidad algoritmica hay que pensar en el peor caso y en esta ocasión
11 contamos con dos while anidados,
12 dichos while's recorren de principio a fin L2 y por cada elemento recorre de principio a fin L1.<br>
13 Entendiendo esto decimos que la complejidad está dada por la siguiente fórmula:  $O(n*m)$ .<br>
14 Por otro lado, si tenemos en cuenta la implementación de listas utilizada en la ejecución, la complejidad no
15 varía, ya que los métodos utilizados provenientes de este TAD es muy similar en las 3 implementaciones que
16 poseemos, además que la complejidad interna de estas sigue siendo  $O(n*m)$ .
```